



University of Tehran
College of Engineering
School of Electrical & Computer Engineering

Experiment 3
Sequential Synthesis and Post Simulation

Digital Logic Laboratory
ECE 045, ECE 895
Laboratory Manual

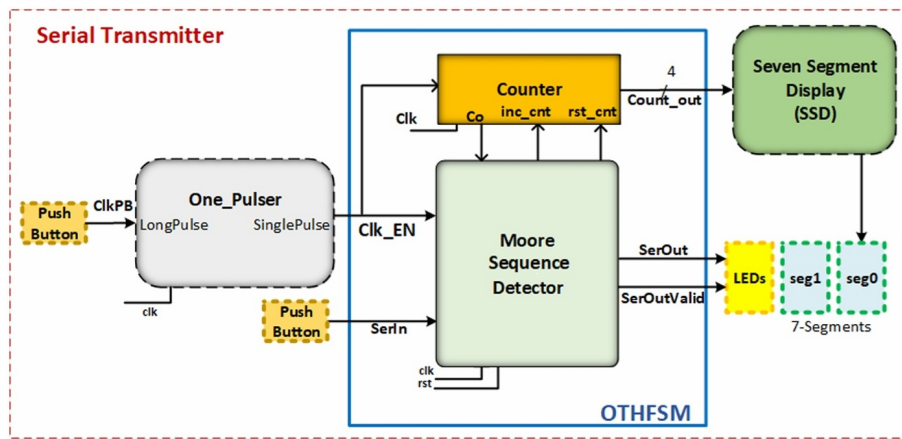
Spring 1405



Contents

Contents	1
Introduction	2
1 Serial Transmitter	2
1.1 Onepulser	2
1.2 Orthogonal Finite State Machine	3
1.3 Seven Segment Display	3
2 Pre-Synthesis Simulation	4
3 Post-Synthesis Simulation	4
Appendices	5
A Using Quartus II	5
A.1 Create the Project	5
A.2 Compilation	5
A.3 Pin Assignment	5
A.4 Program the Design	6
A.5 Examine the Timing and Resources	6
Acknowledgment	6

Figure 1: Serial Transmitter



Introduction

The first goal of this experiment is to introduce the concepts of state machines that are mostly used for controllers. The second goal is to get familiar with FPGA devices and post synthesis simulation. By the end of this experiment, you will learn:

- Concepts of state machines and Sequence detectors
- Huffman coding style
- Pre-synthesis simulation
- Synthesis
- FPGA device and post-synthesis simulation

1 Serial Transmitter

In this experiment you are to design a serial transmitter circuit that searches on its *serIn* input for a start sequence of 1001 to begin transmitting its *serIn* on its *serOut*. When the start sequence is received, the *serOutValid* is asserted and for the next 9 clock cycles whatever appears on *serIn* will be transmitted on *serOut*. After the entire 9 bits are transmitted, the circuit returns to the state where search for the start sequence begins again. The initial state of this circuit is where it searches for the start sequence. The overall design of this experiment is shown in figure 1. As shown some components are needed for this design.

1.1 Onepulser

FPGA clock frequency can vary depending on the device and whether an internal or external clock source is used. In this design, the defined system clock frequency is 25 MHz, which is too fast to

directly monitor the transitions in the module. Therefore, a one-pulser component is required to provide a clock-enable signal for the controller of the design. The *clkEn* input is used to control the effective clock operation when the circuit is implemented on an FPGA board. The one-pulser is connected to a push-button on the board *clkPB*, and each button press generates a single pulse synchronized with the system clock. The output of this circuit connects to the clock enable input *clkEn* of the sequence detector and the counter.

1. Write the Verilog description of the one-pulser module.
2. Test your design in Modelsim.

1.2 Orthogonal Finite State Machine

The second part of this experiment is called an Orthogonal FSM. It consists of a Moore state machine and a counter. When the clock Enable push-button is pushed the Moore sequence detector checks its input and decides which state to go. The sequence detector waits for the sequence of 1001 on its *serIn* input and when this is received, the *serOutvalid* output becomes one. The push button is pressed for every input that the state machine receives. After that, the counter will be enabled and will count for the next 9 consecutive clock cycles. During all these time the *serOutvalid* remains one.

1. Show the state diagram of the sequence detector.
2. Write the top-level Verilog description of the transmitter circuit. In the top-level description, write the Verilog description of the sequence detector and the Verilog description of the counter.
3. Write a testbench for the serial transmitter unit in Verilog that tests the correctness of your circuit and show the results in ModelSim.

1.3 Seven Segment Display

This section is not required for the simulation part of this experiment and is only intended for FPGA hardware implementation. The purpose of this module is to display the number of valid output detections *serOutvalid* on the seven-segment displays embedded on the FPGA board. This part is included to familiarize students with practical FPGA implementation concepts and can be useful for future projects and applications. In this experiment, only simulation is performed; therefore, the seven-segment display module is not implemented in hardware.

The last part of this experiment is a display module used for displaying the counter output on the seven-segment displays of the FPGA board. Each seven-segment display receives a 4-bit input and displays the corresponding hexadecimal (HEX) value on its 7-bit output.

1. Write the Verilog description of the seven segment display module.
2. As an additional point, you can convert the counter output to BCD values and then display it on the seven segments using the SSD Module .

2 Pre-Synthesis Simulation

In this part, you will simulate your pre-synthesis design in Modelsim.

1. Provide all your Verilog codes.
2. Prepare a proper testbench with a 20 MHz clock frequency and generate long push-button pulses with appropriate timing so that the circuit can detect the start sequence and capture the next 9 input bits.
3. Simulate the design and verify the achieved output.
4. Report your result pre-synthesis simulation for finding two detections.

3 Post-Synthesis Simulation

In this part, you will use a Cyclone IV E family device for synthesis. Before synthesizing your design, create a top-level Verilog description that includes both modules (the one-pulser and the FSM) connected together.

1. Make a Quartus project.
2. Add all Verilog codes to your project.
3. Set the the top-level Verilog code as top-level entity by right-click and choose it.
4. Compile and synthesize your design.
5. Use .vo and .sdo files for post- simulation.
6. Prepare a proper testbench with a 20 MHz clock frequency and generate long push-button pulses with appropriate timing so that the circuit can detect the start sequence and capture the next 9 input bits.
7. Report your result post-synthesis simulation for finding two detections.

Appendices

Appendix A Using Quartus II

A.1 Create the Project

1. Click on *File* ▷ *New Project Wizard*
2. Create an appropriate directory for your project and complete the form
3. Select the FPGA device as *Cyclone EP2C20F484C7* and then click *Finish*
4. From *File* ▷ *New* select the *Verilog HDL File*
5. Write your Verilog code in this window

A.2 Compilation

- Select *Processing* ▷ *Start Compilation*
- Click on quick shortcut ►

There may be lots of warnings and some errors after compilation. The warnings are not so important while the errors should be completely removed.

A.3 Pin Assignment

After successfully compiling your design, you should set your design with physical pins of the Cyclone II FPGA on DE1 board. You can find the position and the index of each pin from the DE1 user manual.

Figure 2: Creating new project

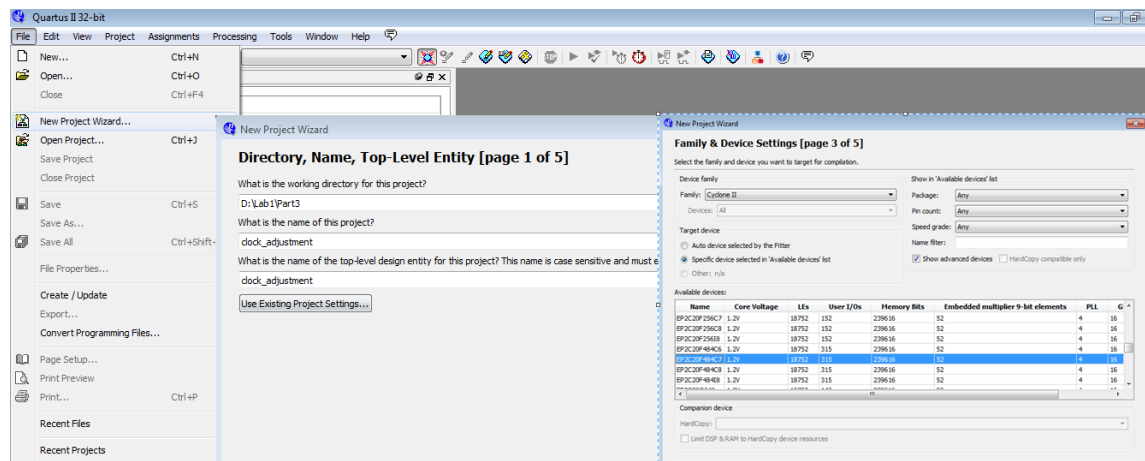
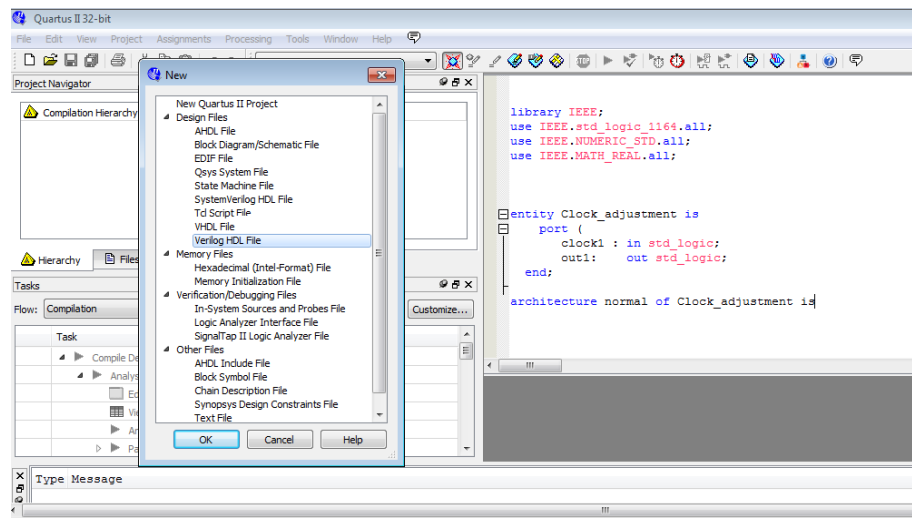


Figure 3: Writing Verilog HDL code



From *Assignments* ▷ *Pin Planner*, **Pin Planner window** will appear and you can select the corresponding locations from the list. When you are done with this, recompile your project.

A.4 Program the Design

1. Click on the  icon on the toolbar to open *Programmer*
2. Choose *USB Blaster* from the *Hardware setup* in the new window
3. Click on the start button

Your design will be programmed on the board when it is finished.

A.5 Examine the Timing and Resources

Now you can examine the resource usage of your design from *Processing* ▷ *Compilation Report* or by clicking on quick shortcut , and the timing from the *Tools* ▷ *TimeQuest Timing Analyzer*.

Acknowledgment

This lab manual was prepared and developed by **Katayoon Basharkhah**, Ph.D. student of Digital Systems at the University of Tehran, under the supervision of Professor Zain Navabi.

This manual has been revised and edited by **Zahra Mahdavi** Ph.D. student of Digital Systems at the University of Tehran, **Zahra Hojati**, and **Iman Rasouli** undergraduate students of Electrical Engineering at the University of Tehran.

Figure 4: Pin Planner window

